

Molnutveckling med Azure (60 yhp)
JENSEN Education

Tillvägagångssätt CI/CD pipeline

(Eller hela kedjan)

Mikael Kratz

Uppsättning av Azure

Jag har följt dessa steg från start till slut:

1. Skapat en resursgrupp för att samla alla resurser.
2. Skapat Application Insights.
3. Skapat Azure Container Registry (ACR) för Docker-images
4. Skapat Container Apps Environment för driftsättning
5. Aktiverat Managed Identity för Container App samt AcrPull-rättighet
6. Lagt till miljövariabler i Container App
 - a. SQL connection string
 - b. Application Insights connection string
 - c. Användarnamn och lösenord för att kunna logga in i appen
7. CI/CD – pipeline
 - a. Skapade ett projekt och repo i Azure DevOps och pushade lokal kod
 - b. Skapade en Service Connection mot ACR för att ge DevOps rättighet att pusha Docker-images till mitt Container Registry
 - c. Pipeline – jag byggde upp min pipeline i tre steg för att kunna felsöka lättare
 - i. En minimal pipeline som bara byggde imagen utan koppling till ACR för att verifiera att bygget fungerade.
 - ii. Bygga och pusha till ACR
 - iii. Deploy till Container App

AZURE CLI

1. Sätt upp variabler i terminalen

2. Skapa Resource Group

```
az group create --name $resourceGroupName`  
--location $location
```

3. Skapa SQL Server

```
az sql server create --name $sqlServer`  
--resource-group $resourceGroupName --location $location`  
--admin-user $adminUser`  
--admin-password $adminPass
```

4. Öppna brandväggen för Azure-tjänster

```
az sql server firewall-rule create --resource-group $resourceGroupName`  
--server $sqlServer --name AllowAzureServices`  
--start-ip-address 0.0.0.0`  
--end-ip-address 0.0.0.0
```

5. Skapa databasen

```
az sql db create --resource-group $resourceGroupName`  
--server $sqlServer --name $databaseName`  
--service-objective Basic
```

6. Spara connection string

```
az sql db show-connection-string`  
--client ado.net`  
--server sql-freakyfashion`  
--name FreakyFashionDb`  
-o tsv
```

Byt ut User ID och Password i det du får tillbaka

7. Skapa Application Insights

```
az monitor log-analytics workspace create`  
--workspace-name $workspace`  
--resource-group $resourceGroupName`  
--location $location
```

```
az monitor app-insights component create`  
--app $appInsight --resource-group $resourceGroupName`  
--location $location`  
--workspace $workspace
```

```
$connectionString = az monitor app-insights component show`  
--app $appInsight --resource-group $resourceGroupName`  
--query "connectionString"`  
--output tsv
```

```
az containerapp update `  
  --name freakyfashion-api `  
  --resource-group rg_freakyfashion `  
  --set-env-vars APPLICATIONINSIGHTS_CONNECTION_STRING=$connectionString
```

8. Skapa Azure Container Registry (ACR)

```
az acr create `  
  --name $containerRegistryName --resource-group $resourceGroupName `  
  --sku Basic `  
  --admin-enabled true
```

Hämta inloggningsuppgifter:

```
az acr credential show `  
  --name $containerRegistryName `  
  --resource-group $resourceGroupName
```

9. Skapa Container Apps Environment

```
az containerapp env create `  
  --name $env --resource-group $resourceGroupName `  
  --location $location
```

10. Skapa projekt i Azure DevOps

- a. Skapa repo
- b. Pusha koden

11. Säkerställ Docker Image lokalt

```
docker build -t freakyfashion-api:latest .
```

12. Skapa koppling till ACR

```
az acr login --name mcraatzacr  
docker tag freakyfashion-api:latest mcraatzacr.azurecr.io/freakyfashion-api:latest  
docker push mcraatzacr.azurecr.io/freakyfashion-api:latest
```

13. Ge Container Apps Environment rättighet till ACR

```
$principalId = az containerapp show `  
  --name $container `  
  --resource-group $resourceGroupName `  
  --query identity.principalId `  
  -o tsv
```

```
$acrResourceId = az acr show --name mcraatzacr --query id -o tsv
```

```
az role assignment create `  
  --assignee $principalId `  
  --role AcrPull `
```

14. Skapa Container App

```
az containerapp create `
--name $container `--resource-group $resourceGroupName`
--environment $env `--image mcratzacr.azurecr.io/freakyfashion-api:latest`
--target-port 8080 `--ingress external`
--min-replicas 1 `--max-replicas 1`
--cpu 0.25 `--memory 0.5Gi`
--registry-server mcratzacr.azurecr.io
```

```
az containerapp identity assign `
--name $container `
--resource-group $resourceGroupName `
--system-assigned
```

15. Sätt miljövariabler

```
az containerapp update `
--name $container `
--resource-group $resourceGroupName `
--set-env-vars `
  ConnectionStrings__DefaultConnection="Server=tcp:sql-
freakyfashion.database.windows.net,1433;Database=FreakyFashionDb;User
ID=xxxxx;Password=xxxxx;Encrypt=True;TrustServerCertificate=False"
```

```
az containerapp revision list `
--name freakyfashion-api `
--resource-group rg_freakyfashion `
-o table
```

```
az containerapp revision restart `
--name freakyfashion-api `
--resource-group rg_freakyfashion `
--revision freakyfashion-api-00000xx
```

```
az containerapp update `
--name freakyfashion-api `
--resource-group rg_freakyfashion `
--set-env-vars `
  Auth__Username="xxxxx" `
  Auth__Password="xxxxx"
```

16. Innan CI/CD - bygga och pusha

```
docker build -t mcratzacr.azurecr.io/freakyfashion-api:20260610-x .  
docker push mcratzacr.azurecr.io/freakyfashion-api:20260610-x
```

```
az containerapp update `  
  --name freakyfashion-api `  
  --resource-group rg_freakyfashion `  
  --image mcratzacr.azurecr.io/freakyfashion-api:20260610-x
```

17. Sätt upp CI/CD

- Azure DevOps -> Pipelines -> Create Pipeline
- Azure Repos Git
- Välj repo: FreakyFashionRepo
- azure-pipelines.yml:

```
trigger:  
- master
```

```
pool:  
  vmImage: 'ubuntu-latest'
```

```
steps:
```

- checkout: self
- script: |
 docker build -t freakyfashion-api:test .
 displayName: 'Build Docker image'

18. Build → Push image till Azure Container Registry

Skapa Service Connection:

- a. Azure DevOps -> Project Settings -> Pipelines -> Service connections
- b. Create
- c. Docker Registry -> Next
- d. Registry type: Azure Container Registry
- e. Service connection:
 - i. Authentication type: Service Principal
 - ii. Azure container registry: mcratzacr
 - iii. Service connection name: mcratzacr
 - iv. Kryssa i: Grant access permission to all pipelines
 - v. Save
- f. Skapa ytterligare en Service Connection för deploy till Container App:
 - i. Azure DevOps -> Project Settings -> Pipelines -> Service connections
 - ii. New -> Azure Resource Manager -> Next
 - iii. Identity type: App registration (automatic)
 - iv. Credential: Workload identity federation
 - v. Scope: din subscription / resource group rg_freakyfiction
 - vi. Service connection name: azure-freakyfiction
 - vii. Kryssa i: Grant access permission to all pipelines
 - viii. Save
 - ix. Azure-pipelines.yaml

trigger:

- master

pool:

vmImage: 'ubuntu-latest'

variables:

imageName: 'freakyfiction-api'

tag: '\$(Build.BuildId)'

steps:

- checkout: self

BUILD (utan registry!)

- task: Docker@2

displayName: Build image

inputs:

command: build

repository: \$(imageName)

dockerfile: Dockerfile

tags: |

\$(tag)

PUSH (kopplar registry via service connection)

```
- task: Docker@2
  displayName: Push image to ACR
  inputs:
    command: push
    repository: $(imageName)
    containerRegistry: 'mcratzacr'
  tags: |
    $(tag)
```

19. Deploy to Container App

Lägg till följande till azure-pipelines.yaml:

```
- task: AzureCLI@2
  displayName: Deploy to Container App
  inputs:
    azureSubscription: 'azure-freakyfashion'
    scriptType: bash
    scriptLocation: inlineScript
    inlineScript: |
      az containerapp update \
        --name freakyfashion-api \
        --resource-group rg_freakyfashion \
        --image mcratzacr.azurecr.io/freakyfashion-api:$(tag)
```